



AI휴먼 과정 · 5일차

과정명

AI휴먼

강의일차

5일차

강의주제

Python 반복문과 컬렉션으로
여러 데이터 처리하기

핵심 키워드

for / while / range / List / Dictionary

오늘의 학습 목표

- 1 for문으로 여러 데이터를 순서대로 처리할 수 있습니다.
- 2 while문으로 조건이 참인 동안 반복할 수 있습니다.
- 3 range()로 반복 횟수와 숫자 범위를 만들 수 있습니다.
- 4 List에 여러 값을 저장하고 반복 처리할 수 있습니다.
- 5 Dictionary에 이름과 값의 관계를 저장하고 반복 처리할 수 있습니다.

✔ 오늘의 산출물: 학생 성적 반복 처리 프로그램

A close-up photograph of a heavy metal chain resting on a wooden surface. The chain is made of large, interlocking links and runs diagonally from the top left towards the bottom right. The wood grain is clearly visible, and the lighting creates a soft glow on the right side of the image.

전날 학습 연결: 조건문

전날 핵심

if / elif / else / Boolean

조건문은 "어떤 경우에 무엇을 할 것인가"를 결정합니다.

반복문은 "같거나 비슷한 처리를 여러 번 수행"합니다.

두 문법을 함께 사용하면 여러 데이터를 조건에 따라 분류할 수 있습니다.

예: 학생 4명의 점수를 하나씩 확인해
PASS 또는 RETRY 출력

왜 반복문이 필요한가?

반복문이 없을 때

```
print(scores[0])  
print(scores[1])  
print(scores[2])  
print(scores[3])
```

반복문을 사용할 때

```
for score in scores:  
    print(score)
```

📌 핵심 메시지: 데이터 개수가 늘어날수록 반복문은 코드 중복을 크게 줄여 줍니다.

반복(Iteration)이란?

Iteration: 같은 형태의 작업을 순서대로 되풀이하는 처리

Python의 대표 반복문:

for

여러 값 또는 정해진 범위를 순회할 때 적합

while

특정 조건이 참인 동안 계속 실행할 때 적합

반복 처리의 기본 구조: 입력 데이터 → 하나씩 꺼내기 → 처리 → 결과 누적 또는 출력

for문의 기본 구조

문법

```
for 변수 in 반복가능한데이터:  
    반복할 코드
```

예제

```
scores = [72, 88, 95, 61]  
for score in scores:  
    print(score)
```

해석

scores의 값을 앞에서부터 하나씩 score에 넣고 print()를 실행합니다.

for문 의사코드

01

점수 목록을 준비한다.

02

점수 목록에서 점수를 하나 꺼낸다.

03

꺼낸 점수를 출력한다.

04

아직 남은 점수가 있으면 다시 하나 꺼낸다.

05

모든 점수를 처리하면 반복을 종료한다.

📌 핵심: for문에서는 "데이터를 하나씩 꺼내는 작업"을 Python이 자동으로 처리합니다.

range() 함수

range(): 연속된 정수 범위를 만들어 반복 횟수를 제어할 때 사용하는 함수

range(끝)

range(시작, 끝)

range(시작, 끝, 증가값)

⚠️ 중요: 끝 값은 포함하지 않습니다. 예: range(1, 5) → 1, 2, 3, 4

range() 예제

예제 1

```
for i in range(5):  
    print(i)
```

결과: 0, 1, 2, 3, 4

예제 2

```
for i in range(1, 6):  
    print(i)
```

결과: 1, 2, 3, 4, 5

예제 3

```
for i in range(2, 11, 2):  
    print(i)
```

결과: 2, 4, 6, 8, 10

while문의 기본 구조

문법

```
while 조건식:  
    반복할 코드
```

예제

```
count = 1  
while count <= 5:  
    print(count)  
    count += 1
```

해석

count <= 5가 True인 동안 코드 블록을 반복합니다.

while문 의사코드

01

count를 1로 준비한다.

03

참이면 count를 출력한다.

05

다시 조건을 확인한다.

02

count가 5 이하인지 확인한다.

04

count를 1 증가시킨다.

06

조건이 거짓이 되면 반복을 종료한다.

⚠ 주의: 반복 조건이 계속 True라면 반복이 끝나지 않는 무한 반복이 발생할 수 있습니다.

for와 while 비교

for가 적합한 경우

- 리스트의 모든 값을 하나씩 처리
- 1부터 10까지 정해진 횟수 반복
- 데이터 개수가 정해져 있거나 순회 대상이 분명할 때

while이 적합한 경우

- 사용자가 종료를 선택할 때까지 반복
- 특정 조건을 만족할 때까지 반복
- 반복 횟수보다 종료 조건이 더 중요한 경우

List란?

List(리스트): 여러 값을 순서대로 저장하는 Python 컬렉션

표현 방법: 대괄호 []

```
scores = [72, 88, 95, 61]
names = ["민수", "지연", "준호"]
```

특징

- 값의 순서가 있습니다.
- 0부터 시작하는 인덱스로 값에 접근합니다.
- 값을 추가·수정·삭제할 수 있습니다.

List의 인덱스

```
scores = [72, 88, 95, 61]
```

인덱스 0

72

인덱스 1

88

인덱스 2

95

인덱스 3

61

```
print(scores[0]) # 72  
print(scores[-1]) # 61
```

📌 핵심: 반복문을 사용하면 인덱스를 직접 하나씩 작성하지 않고 전체 값을 순차 처리할 수 있습니다.

List와 for문

```
scores = [72, 88, 95, 61]

for score in scores:
    if score >= 80:
        print(score, "PASS")
    else:
        print(score, "RETRY")
```

학습 연결

List → 여러 점수 저장

for → 점수 하나씩 꺼내기

if → 합격 여부 판단

누적 계산 패턴

여러 값의 합계를 구하는 기본 패턴:

```
total = 0
for score in scores:
    total += score

average = total / len(scores)
```

📌 용어 — Accumulator(누적 변수): 반복하면서 계산 결과를 계속 저장하는 변수. 예: total, count, highest

합격자 수 세기

```
pass_count = 0

for score in scores:
    if score >= 80:
        pass_count += 1

print("합격자 수:", pass_count)
```

📌 핵심: 조건을 만족할 때만 **count** 값을 증가시키는 형태는 데이터 분석에서도 자주 사용합니다.

최고점 직접 찾기

```
scores = [72, 88, 95, 61]  
highest = scores[0]
```

```
for score in scores:  
    if score > highest:  
        highest = score
```

```
print("최고점:", highest)
```

의미

현재 최고점과 새 점수를 비교하면서 더 큰 값을 저장합니다.

Dictionary란?

Dictionary(딕셔너리): Key와 Value의 관계로 데이터를

저장하는 컬렉션

표현 방법: 중괄호 { }

```
students = {  
    "민수": 72,  
    "지연": 88,  
    "준호": 95  
}
```

Key

값을 찾기 위한 이름

Value

Key에 연결된 실제 데이터

Dictionary 값 읽기

```
students = {"민수": 72, "지연": 88, "준호": 95}
```

```
print(students["지연"])
```

결과: 88

keys()

Key들을 확인

values()

Value들을 확인

items()

Key와 Value를 함께 확인

오늘은 items()를 사용해 이름과 점수를 함께 반복 처리합니다.

Dictionary와 for문

```
students = {  
    "민수": 72,  
    "지연": 88,  
    "준호": 95  
}  
  
for name, score in students.items():  
    print(name, score)
```

해석

반복할 때마다 학생 이름은 `name`에, 점수는 `score`에 저장됩니다.

조건문 + 반복문 + Dictionary

```
for name, score in students.items():  
    if score >= 90:  
        grade = "A"  
    elif score >= 80:  
        grade = "B"  
    elif score >= 70:  
        grade = "C"  
    else:  
        grade = "D"  
  
    print(name, score, grade)
```

대표 실습 프로그램의 입력 → 처리 → 출력

입력

학생 이름과 점수가 저장된 Dictionary

출력

학생별 결과표 + 평균 + 합격자 수 + 최고점

처리

1. items()로 학생별 데이터 반복
2. 점수에 따라 등급 판정
3. 합계 누적
4. 합격자 수 계산
5. 최고점 학생 찾기

대표 실습 의사코드

01

학생 이름과 점수를 딕셔너리에 저장한다. 합계, 합격자 수를 0으로 준비한다.

02

03

최고점과 최고점 학생을 준비한다.

04

학생 정보를 하나씩 반복한다.

05

점수를 합계에 더한다.

06

80점 이상이면 합격자 수를
1 증가시킨다.

07

현재 점수가 최고점보다 높으면
최고점 정보를 바꾼다.

08

조건문으로 등급을 결정한다.

09

학생 이름, 점수, 등급을 출력한다.

10

평균을 계산한다.

11

전체 요약 결과를 출력한다.

자주 발생하는 오류

오류 1. 들여쓰기 누락

```
for score in scores:  
print(score)
```

→ 반복문 내부 코드는 들여쓰기 필요

오류 2. while의 증가식 누락

```
while count <= 5:  
    print(count)
```

→ count가 변하지 않아 반복이 종료되지 않음

오류 3. range()의 끝 값 착각

range(1, 5) → 1, 2, 3, 4

오류 4. Dictionary에 존재하지 않는 Key 사용

students["서연"] → KeyError 가능

디버깅 체크리스트

- 1 for 또는 while 다음에 콜론(:)이 있는가?
- 2 반복문 내부 코드가 같은 수준으로 들여쓰기 되었는가?
- 3 while 조건을 언젠가 False로 만드는 코드가 있는가?
- 4 range()의 시작·끝·증가값이 의도와 맞는가?
- 5 List 인덱스가 0부터 시작한다는 점을 확인했는가?
- 6 Dictionary의 Key 이름이 정확한가?
- 7 누적 변수 total, count의 초기값이 반복문 밖에 있는가?



오늘의 3단계 과제

Basic

점수 List의 값을 순서대로 출력하고
PASS/RETRY 표시

Application

점수 List를 반복하면서
합격자 수·평균·최고점 계산

Challenge

학생 이름과 점수를 Dictionary로 구성하고 학생별 등급표와 전체 요약 출력



과제 원칙: 입력 → 반복 → 조건 → 누적 → 출력의 흐름을
설명할 수 있어야 합니다.

오늘의 핵심 정리와 다음 학습



for

데이터를 하나씩 순회



while

조건이 참인 동안 반복



range

반복에 사용할 정수 범위 생성



List

순서가 있는 여러 값 저장



Dictionary

Key-Value 관계로 여러 값 저장



다음 학습: 반복되는 코드를 함수(Function)로 묶고 매개변수(Parameter)와 반환값(return)을 사용합니다.